

The Limitless Stack

A complete AI operating protocol — clone, install, run.

The Limitless Stack is seven tools wired into one operating system: **Claude**, **CLAUDE.md**, **Obsidian**, **NotebookLM**, **Pinecone**, **Hub Workspace**, and **Paperclip**. Each tool is bad at the others' jobs. Together they form a memory-and-coordination layer that makes a small team operationally viable across many projects.

This guide takes you from *git clone* to a working stack on your own machine in roughly 30–45 minutes. Read it once before you start; paste it into your Claude session as the onboarding document so your Claude inherits the same operating discipline that ours does.

What you'll have when you're done

- **A working knowledge base** in Obsidian that compounds — every new source updates many pages, never derived from scratch.
- **Semantic recall** across every repo and document via Pinecone — natural-language search, no grep gymnastics.
- **Curated topic notebooks** in NotebookLM — deep research over source collections, plus a special *reminder bucket* your Claude reads at the start of every session.
- **Mechanical session rituals**: Roll Call at start, an 8-step checklist at end. Both bundled as installable skills.
- **A living anti-patterns log** — institutional memory of mistakes worth not repeating, ships from day one.
- **Six skills** auto-installed to `~/claude/skills/`: limitless-stack, notebooklm, four-tool-lookup, roll-call, verify-before-claim, karpathy-guidelines.

The seven tools

Each tool plays one role in the larger system. None of them is novel on its own; the protocol is in how they're connected.

Tool	Role	What it owns
Claude	Reasoning engine	Reads, writes, connects, decides
CLAUDE.md	Identity + rules	Trust anchor, voice, guardrails per context
Obsidian	Knowledge base	Structured wiki: architecture, decisions, entities
NotebookLM	Research desk	Deep research across curated source collections
Pinecone	Semantic memory	Full-text recall across every repo and document
Hub Workspace	Multi-model agent runtime	Chat surface — Gemini default, Claude opt-in
Paperclip	Coordination	Org chart, budgets, tickets, approvals

One integrated system, one infrastructure. Everything runs on Supabase + Vercel. The same shared Postgres backs every app in the ecosystem. The wiki lives in git (Obsidian + a private GitHub repo). Pinecone and NotebookLM are SaaS endpoints. The Hub Workspace ties them together as a single browser surface.

Why this exists — capabilities you didn't have before

AI assistants have an amnesia problem: every session starts cold. Long contexts balloon costs and degrade reasoning. Classic RAG (chunks → answer → forget) doesn't compound. The Limitless Stack solves this by giving Claude a persistent memory layer it can read AND write — knowledge that gets smarter every time you use it.

Concrete capabilities the Stack unlocks

Knowledge that compounds. One new source can update fifteen wiki pages. Each query is informed by every prior ingest. The wiki gets sharper over time instead of going stale.

Mechanical session rituals. Roll Call (preflight) verifies all seven tools are present and in sync before substantive work starts. The end-of-session checklist guarantees nothing is left uncommitted, unsynced, or unverified. Both are bundled as installable skills.

Cross-repo semantic memory. Pinecone indexes every repo and document. "How does JWT refresh work in our codebase?" surfaces the right files in seconds. No grep, no hunting.

Topic-specific deep research. Per-project NotebookLM notebooks let you query a curated source set with full context — useful when you need a synthesis across many docs that the wiki doesn't yet cover.

The reminder bucket — Claude's session-start context. A special NotebookLM notebook with five curated files (CLAUDE.md, anti-patterns, the seven-tool concept page, the agent-runtime spec, the flagship app page). Your Claude reads it as the FIRST thing each session. Skipping it is the #1 anti-pattern.

Living anti-patterns log. When a session catches a recurring failure mode, it lands in `wiki/synthesis/claude-anti-patterns.md` as a numbered entry. Future Claude sessions read these via the reminder bucket. The system gets smarter from each mistake.

Self-healing infrastructure (when wired up). CLAUDE.md as trust anchor + sandboxed agent execution + diagnose-before-repair discipline lets autonomous fixes run on the right repos with the right guardrails. Multi-fix-candidate dispatch via Hub Workspace gives operators options.

Prerequisites

Accounts you need

- **GitHub** with access to the org whose repos you'll index. A Personal Access Token with repo scope if you don't already have CLI auth.
- **Pinecone** (free Starter tier is fine). Grab the API key from the Pinecone console.
- **Google account with NotebookLM access** — you'll need to be logged into it in Chrome.
- **Invitation to Open-Scaffold-Labs/LimitlessStack**. Matt invites you as a collaborator; you accept.

Software on the Mac

- **Homebrew** (`brew --version`)
- **Python 3.11** (`brew install python@3.11`)
- **Node.js** (`brew install node`)
- **Obsidian** (`brew install --cask obsidian`)
- **gh CLI** (`brew install gh`) — then `gh auth login`
- **Chrome** with the Claude for Chrome extension installed and connected

What you don't get from us

Just so it's clear: cloning the LimitlessStack repo gives you the *protocol* — skills, tools, scaffolding, foundational reading. It does **not** give you access to our actual NotebookLM notebooks, our wiki content, our Hub at `limitless-stack-hub.vercel.app`, our Pinecone index, or any of our private repos. Those are gated by separate authentication. Bucket IDs in the routing config are shown as worked examples — you'll create your own and edit the config to point at yours during install.

Step-by-step install

Run these in order. Each step has a verification check; don't move on until that step passes.

1. Clone the LimitlessStack repo

Once Matt has invited you to `Open-Scaffold-Labs/LimitlessStack`:

```
$ gh repo clone Open-Scaffold-Labs/LimitlessStack ~/LimitlessStack
$ cd ~/LimitlessStack
```

Verify: `ls skills/` shows six skill directories.

2. Run the installer

```
$ ./install.sh ~/your-vault-path
```

This installs all six skills to `~/claude/skills/`, copies the vault template (including the four foundational pages), copies operational tools to your vault's `tools/` directory, and seeds your `CLAUDE.md` from the schema template.

Verify: `ls ~/claude/skills/` shows the six skills installed. `ls ~/your-vault-path/wiki/` shows `index.md`, `log.md`, `overview.md`, plus the foundational pages.

3. Set up Pinecone

```
$ security add-generic-password -a pinecone -s pinecone-api-key -U -w
# (paste your Pinecone API key when prompted)

$ cd ~/your-vault-path
$ python3.11 tools/pinecone-sync.py --dry-run
$ python3.11 tools/pinecone-sync.py
```

Verify: `python3.11 tools/pinecone-search.py "any test query"` returns results with file paths and scores ≥ 0.7 .

4. Create your NotebookLM buckets

In Chrome (signed into NotebookLM), create two notebooks at minimum: a **Wiki default** (upload everything in `wiki/`) and a **Reminder bucket** (upload exactly five files: `CLAUDE.md`, `wiki/synthesis/claude-anti-patterns.md`, `wiki/concepts/llm-wiki-pattern.md`, `wiki/concepts/notebooklm-workflow.md`, and your flagship app page).

Note each notebook's ID. Per-project notebooks (one per significant repo) are optional but recommended.

5. Wire your bucket IDs into the config

Edit `tools/notebooklm-wiki-refresh.py`. At the top, find `NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, and `REMINDER_FILES`. Replace Matt's bucket IDs with yours.

Also edit `tools/limitless-preflight.sh` if it references specific bucket IDs in its dedupe-sweep block.

Verify: `python3.11 tools/notebooklm-wiki-refresh.py --dry-run` shows the wiki files routing to YOUR notebook IDs, not Matt's.

6. Customize for your domain

Open `~/your-vault-path/CLAUDE.md` and replace `[YOUR DOMAIN HERE]` placeholders with your project's name. The vault template's foundational pages reference `[[entities/openscaffold]]` as a worked example — either replace those wiki-links with your own org/project, or leave them and add your own entity pages alongside.

7. Run Roll Call

```
$ bash tools/limitless-preflight.sh
$ echo "exit code: $?"
```

Exit 0 = READY (all seven tools verified). Exit 1 = WARN (proceed with caveats). Exit 2 = BLOCK (fix before substantive work).

Your first session with Claude

Open a Claude Code or Cowork session in your new vault directory. The limitless-stack skill is now installed and Claude will read CLAUDE.md automatically.

Prompt to paste in

Paste this exactly to bootstrap a session aligned to the protocol:

```
Hey Claude. This vault uses the Limitless Stack protocol. Run roll-call,
then bootstrap, then query my reminder NotebookLM bucket for the current
operating rules and recent anti-patterns. Then read wiki/index.md.
Then we'll start work.
```

What Claude should do (and verify)

Invoke roll-call It runs `tools/limitless-preflight.sh`. Returns READY/WARN/BLOCK.

Run session-bootstrap.sh Snapshot of wiki state, Pinecone vector count, recent log entries, contradictions.

Query the reminder NotebookLM bucket `notebooklm use <your-reminder-id> && notebooklm ask "What are the current operating rules? What anti-patterns should I avoid?"`

Read wiki/index.md Catalog of every page. Know what exists before answering anything.

Apply the four-tool lookup order to every question Wiki → Pinecone → NotebookLM (if deep research needed) → only then reason from context.

End-of-session checklist (your Claude must run this)

When you wrap up, your Claude runs the eight-step checklist. Steps 6 and 7 are load-bearing — they exist because past sessions left wiki work uncommitted and discovered NotebookLM refreshes were silently no-ops for weeks.

- **0.** Update `wiki/team-tasks.md` + `wiki/my-tasks/<your-login>.md`.
- **1.** Commit and push the wiki repo.
- **2.** Push any other repos modified.
- **3.** Verify `wiki/log.md` has entries for everything done.
- **4.** Run `python3.11 tools/pinecone-sync.py --changed-only`.
- **5.** Run `python3.11 tools/notebooklm-wiki-refresh.py`.
- **6. VERIFY the refresh actually landed** — check the script reports `verify_failed: 0`; if not, query NotebookLM directly.

- **7.** If reminder allowlist files changed, run one explicit query against the reminder bucket and confirm the answer reflects the edit.

Day-to-day operating

Ingest a new source

Drop the file into `raw/`. Then ask your Claude: *"Ingest the new source at raw/{filename}."* Claude reads it, creates `wiki/sources/<slug>.md`, updates any entity/concept pages it touches, flags contradictions, updates `wiki/index.md`, and appends a log entry. One source per request — don't batch unless you've thought about it.

Ask a question

Just ask. Claude will follow the four-tool lookup: read `wiki/index.md`, drill into relevant pages, fall back to Pinecone semantic search if the wiki is thin, escalate to NotebookLM for deep research. Substantive answers get filed back as `wiki/synthesis/<slug>.md`.

Periodic lint

Every few weeks: *"Run a lint pass and write the report."* Surfaces contradictions, stale claims, orphan pages, missing entities, broken links, coverage gaps, index drift. Report lands at `wiki/synthesis/lint-YYYY-MM-DD.md`.

Catch a new failure mode

When you (or Claude) catches a recurring mistake worth not repeating, append a numbered entry to `wiki/synthesis/claude-anti-patterns.md`. Format: trigger pattern, why-it's-tempting, why-it's-wrong, the corrective rule. Future sessions read this via the reminder bucket and learn from your mistake.

Schedule a nightly Pinecone sync

Use `crontab -e` or your scheduler of choice to run `tools/pinecone-sync.py --changed-only` overnight. Keeps the index current without manual ceremony. Skip when the index is over its monthly quota — the script bails on 429s rather than silently corrupting state.

Customization checklist — disconnect from our setup

When you clone, you're getting Matt's working setup as a worked example. To make it yours, here's the find/replace list. Run through this once, end-to-end.

Bucket IDs (operational)

- `tools/notebooklm-wiki-refresh.py` — `NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, `REMINDER_FILES`.
- `tools/limitless-preflight.sh` — any block that references specific bucket IDs in dedupe sweeps.
- `tools/notebooklm-dedupe.py` — usage examples reference our bucket IDs as defaults; replace with yours.

Naming (cosmetic)

- `CLAUDE.md` — replace `[YOUR DOMAIN HERE]` placeholders with your project name.
- `wiki/concepts/llm-wiki-pattern.md` — wiki-links to `[[entities/openscaffold]]` can stay as worked examples or be replaced with your org page.
- `wiki/sources/claude-code-karpathy-obsidian-video-2026-04-14.md` — same; this is the source summary, leave or repoint.
- `wiki/concepts/notebooklm-workflow.md` — references the seven-bucket routing pattern with our notebook IDs as concrete examples; updating these to yours is helpful but optional.
- `wiki/synthesis/claude-anti-patterns.md` — entries reference OpenScaffold-specific tools (e.g. the Hub) as concrete examples. The lessons generalize; the names don't need to.

Repos to clone into raw/

The setup-guide's example clone list is OpenScaffold-specific. Replace with your own repos:

```
$ cd ~/your-vault-path/raw/
$ gh repo clone your-org/your-flagship-repo
$ gh repo clone your-org/another-repo
```

What to keep as-is

- All six skills (`limitless-stack`, `notebooklm`, `four-tool-lookup`, `roll-call`, `verify-before-claim`, `karpathy-guidelines`) — generic, no customization needed.
- `tools/session-bootstrap.sh` — generic, reads from your vault automatically.

- `tools/pinecone-sync.py` + `tools/pinecone-search.py` — generic, read from Keychain and your vault paths.
- `tools/limitless-preflight.sh` base structure — only the bucket-ID dedupe block needs editing.
- The vault directory structure (`raw/`, `wiki/`, `tools/`) — exactly the convention the protocol assumes.

Reference

The six bundled skills

Skill	What it does
<code>limitless-stack</code>	The umbrella protocol. Mandatory first action, end-of-session checklist, reminder bucket, anti-patterns, etc.
<code>notebooklm</code>	Full NotebookLM CLI API. Create notebooks, upload sources, generate artifacts, download results.
<code>four-tool-lookup</code>	Discipline for the wiki → Pinecone → NotebookLM lookup order, separated as its own skill so it's enforced.
<code>roll-call</code>	Session-start preflight. Mechanically verifies all seven tools are present, authenticated, and in sync. Re-runs if not.
<code>verify-before-claim</code>	Guards against false 'unavailable' claims. Before declaring a tool inaccessible, verify across execution environment.
<code>karpathy-guidelines</code>	Behavioral guidelines for code edits. Four named principles: Think Before Coding, Simplicity First, Surge, etc.

Operational tools

Tool	What it does
<code>limitless-preflight.sh</code>	The script Roll Call calls. Validates all seven tools, dedupe-sweeps every NotebookLM bucket, re-runs if not.
<code>session-bootstrap.sh</code>	Current-thesis snapshot at session start. Wiki state, Pinecone count, recent log entries.
<code>notebooklm-wiki-refresh.py</code>	Routes wiki files into per-project + default + reminder NotebookLM buckets. Verifies content actually uploaded.
<code>notebooklm-dedupe.py</code>	Sweep any bucket for duplicate sources. Used when the routing logic produces ghost duplicates.
<code>pinecone-sync.py</code>	Chunks the corpus, upserts to the index. Supports --changed-only, --dry-run, --repo NAME. Rate-limited.
<code>pinecone-search.py</code>	Semantic search across the index with optional --repo filter.

Foundational reading (in `obsidian/vault-template/wiki/`)

- `concepts/llm-wiki-pattern.md` — Karpathy's LLM Wiki pattern. The methodology this vault is built on.
- `sources/claude-code-karpathy-obsidian-video-2026-04-14.md` — Jack Roberts video that distilled the four-tool framing.
- `concepts/notebooklm-workflow.md` — The seven-bucket routing pattern, dedupe operational discipline.
- `synthesis/claude-anti-patterns.md` — Living log of failure modes worth not repeating. Starts with eleven entries; you'll add yours.

Where to go for more

- `skills/limitless-stack/SKILL.md` — full protocol, every section, 400+ lines.
- `docs/setup-guide.md` — detailed step-by-step (this PDF is the highlight reel; the setup guide is the full reference).
- `README.md` — what's in the repo, organized by section.
- `CLAUDE.md` (repo root) — repo-specific rules.
- Upstream — `forrestchang/andrej-karpathy-skills` for the `karpathy-guidelines` source. MIT licensed.

The Limitless Stack is one integrated system. Don't try to use just one tool — the value compounds across all seven. Welcome aboard.